



APPLIED MATH FOR COMPUTER SCIENCE

CPCS 212

LAB 3 : MATLAB programming

Agenda

- **Input/output commands**
- X-Y Plot (2D Graph)
- Select a plot region

Lab 3

Open the diary file
`Lab3_yourname.txt`

Input/output commands

Command	Description	Example	Result
<code>x= input('text')</code>	Displays the text in quotes, waits for user input from the keyboard, and stores the value in x.	<code>>> x = input('Enter your ID: ')</code>	Enter your ID :321 <code>x = 321</code>
<code>x= input('text' , 's')</code>	Displays the text in quotes, waits for user input from the keyboard, and stores the input as a string in x.	<code>>> y = input('Enter your name: ','s')</code>	Enter your name: Huda <code>y = Huda</code>
<code>disp('text')</code>	Displays the text string enclosed within quotes.	<code>>> disp(' Hello world')</code>	Hello world
<code>disp(a)</code>	Displays the contents, but not the name of the array A.	<code>>> a= 2012</code> <code>>> disp(a)</code>	2012

Input/output commands

Command	Description
fprint	Performs formatted writes to the screen or to a file
Escape characters	\b Backspace \f Form feed %% Percent character \n New line \\ Backslash \r Carriage return \xN Hexadecimal number N \t Horizontal tab \N Octal number N " Single quotation mark

The *outFormat* string specifies how the *outVariables* are converted and displayed. The *outFormat* string can contain any text characters. It also must contain a *conversion code* for each of the *outVariables*. The following table shows the basic conversion codes.

Code	Conversion instruction
%s	format as a string
%d	format with no fractional part (integer format)
%f	format as a floating-point value
%e	format as a floating-point value in scientific notation
%g	format in the most compact form of either %f or %e
\n	insert newline in output string
\t	insert tab in output string

In addition to specifying the type of conversion (e.g. %d, %f, %e) one can also specify the width and precision of the result of the conversion.

Syntax:

%wd
%w.pf
%w.pe

where *w* is the number of characters in the width of the final result, and *p* is the number of digits to the right of the decimal point to be displayed.

Input/output commands

Formatted String Conversion Specifications	<code>%d</code> or <code>%i</code>	Signed Integer	<code>%f</code>	Floating-point number
	<code>%u</code>	Unsigned Integer	<code>%e</code>	Exponential notation
	<code>%o</code>	Octal	<code>%c</code>	Single character
	<code>%x</code>	Hexadecimal	<code>%s</code>	String of characters

```
>> fprintf('a\bb\n')
fprintf('a\tb\n')
fprintf('a\nb\n')
fprintf('a = %d\n',5)
fprintf('a = %5d\n',5)
fprintf('a = %f\n',5.5)
fprintf('a = %6.3f\n',5.5)
fprintf('a = %s\n','5')
b
a   b
a
b
a = 5
a =      5
a = 5.500000
a =  5.500
a = 5
```

Functions for String Manipulation (1)

Function	Operation
<code>char</code>	Converts an integer to the character using ASCII codes, or combines characters into a character matrix
<code>findstr</code>	Finds one string in another string
<code>length</code>	Returns the number of characters in a string
<code>num2str</code>	Converts a number to string
<code>str2num</code>	Converts a string to a number
<code>strcmp</code>	Compares two strings
<code>strmatch</code>	Identifies rows of a character array that begin with a string
<code>strncmp</code>	Compares the first n elements of two strings
<code>sprintf</code>	Converts strings and numeric values to a string

Lab 3

Exercise 1

1-(a)

Execute the following simple loop.

```
>> % Exercise 1:  
>> % 1 - (a)  
>>for i=0:12  
    disp(['The square root of ',  
num2str(i^2), ' is ' , num2str(i)]);  
end
```

1-(a)

The square root of 0 is 0

The square root of 1 is 1

The square root of 4 is 2

The square root of 9 is 3

The square root of 16 is 4

The square root of 25 is 5

The square root of 36 is 6

The square root of 49 is 7

The square root of 64 is 8

The square root of 81 is 9

The square root of 100 is 10

The square root of 121 is 11

The square root of 144 is 12

1-(b)- homework

Note that the “*strcat*” function was not used in this code and string concatenation is accomplished by generating a row vector. What would the output look like if *strcat* were used?

```
for i=0:12
```

```
x=strcat('The square root of ',num2str(i^2),' is ' , num2str(i));
```

```
    x
```

```
end
```

1-(c)

Re-write the loop using *fprintf* so that the numbers that are printed are aligned.

1-(c)

- Re-write the loop using *fprintf* so that the numbers that are printed are aligned.
- Please include a copy of your *fprintf* command in a file.

```
>> for i = 0:12  
fprintf ('The square root of   %3d   is   %3d \n',i^2,i);  
end
```

1-(c)

The square root of	0	is	0
The square root of	1	is	1
The square root of	4	is	2
The square root of	9	is	3
The square root of	16	is	4
The square root of	25	is	5
The square root of	36	is	6
The square root of	49	is	7
The square root of	64	is	8
The square root of	81	is	9
The square root of	100	is	10
The square root of	121	is	11
The square root of	144	is	12

1-(d) - Homework

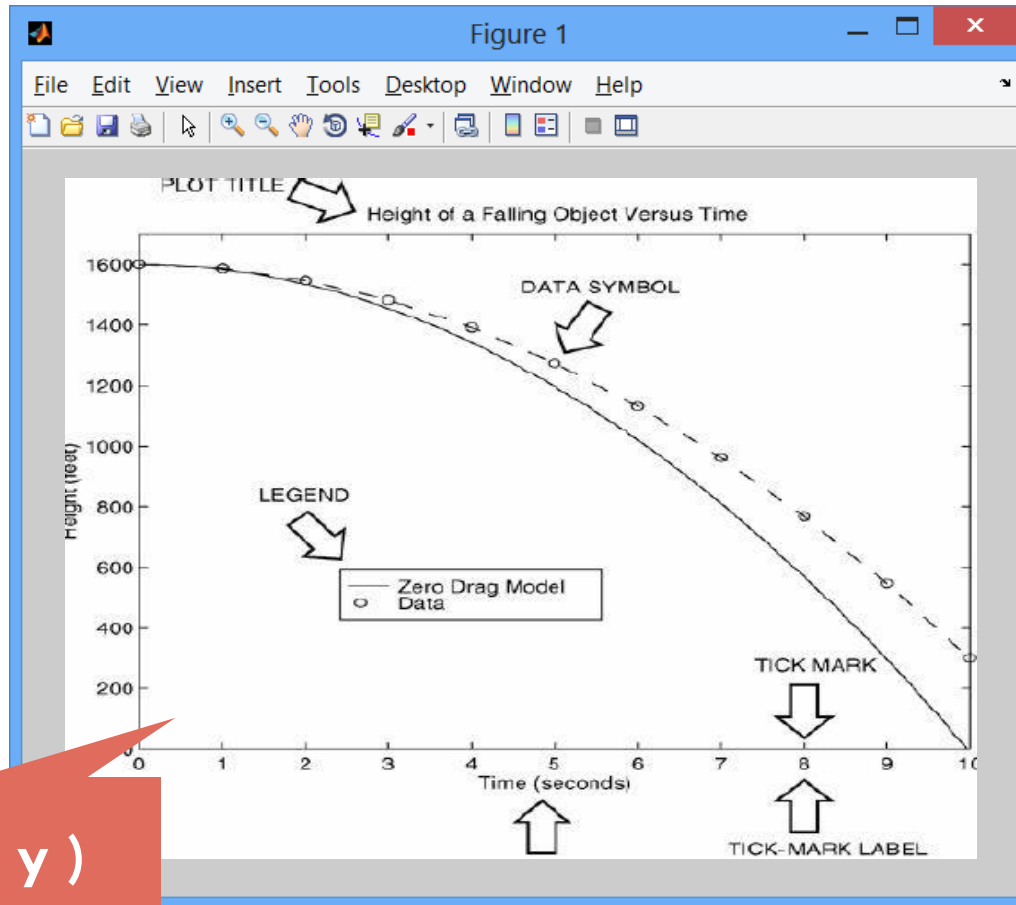
Replace the square root with the tangent, so that the tangents of the numbers 0, 1, 4, 9... 144 are printed and so the decimal points are aligned.

1-(e) - Homework

Similarly, replace the tangent with the exponential, so that the exponentials of the numbers 0, 1, 4, 9... 144 are printed and so the decimal points are aligned.

X-Y Plot (2D Graph)

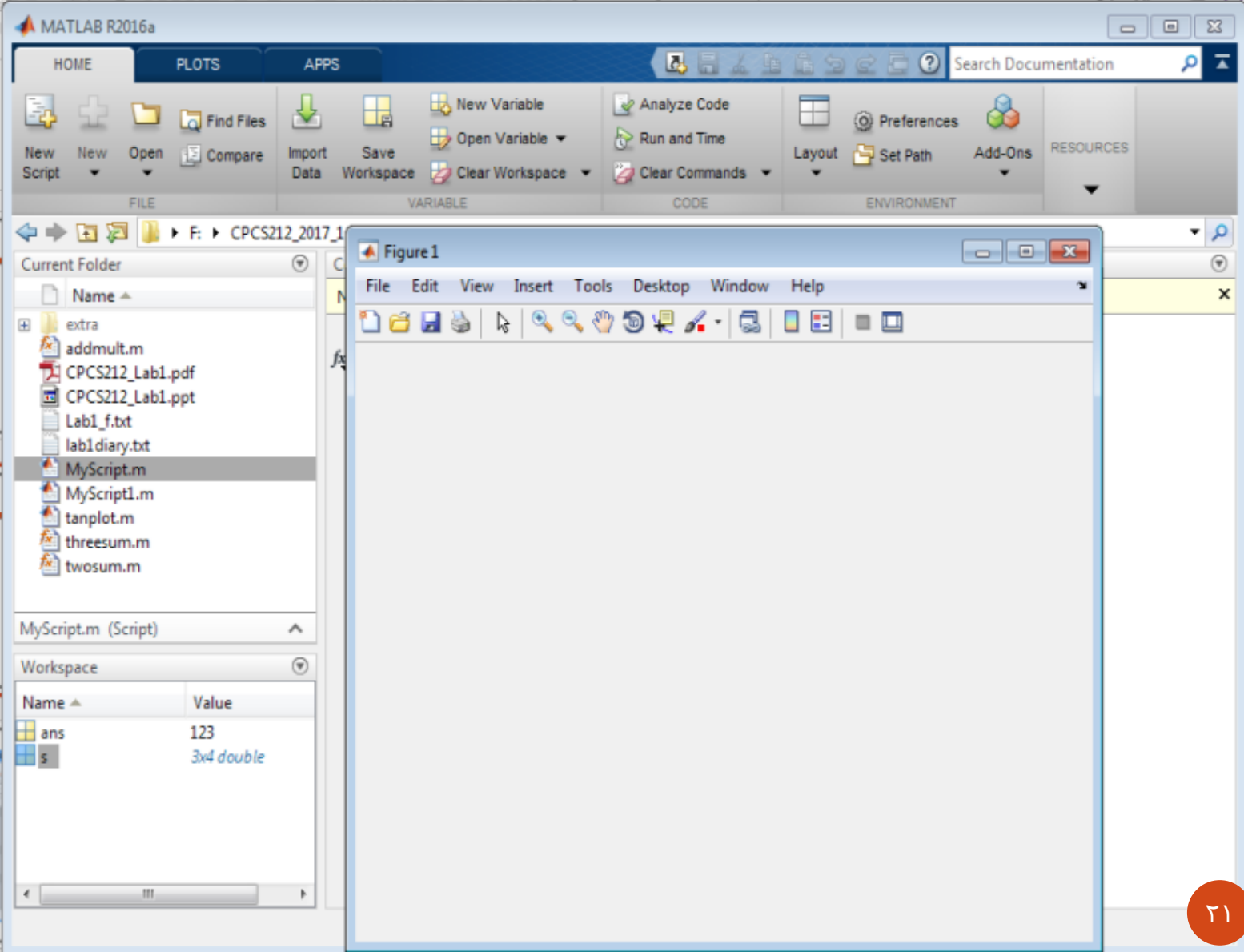
Figure



Plot(x , y)

Introduction to MATLAB Graph

- The MATLAB environment offers a variety of data plotting functions plus a set of GUI tools to create, and modify graphic displays. The GUI tools afford most of the control over graphic properties and options that typed commands such as `annotate`, `get`, and `set` provide.
- A *figure* is a MATLAB window that contains graphic displays (usually data plots) and UI components. You create figures explicitly with the **"figure"** function, and implicitly whenever you plot graphics and no figure is active. By default, figure windows are resizable and include pull-down menus and toolbars.



Introduction to MATLAB Graph

- A *plot* is any graphic display you can create within a figure window. Plots can display tabular data, geometric objects, surface and image objects, and annotations such as titles, legends, and colorbars. Figures can contain any number of plots. Each plot is created within a 2-D or a 3-D data space called an *axes*. You can explicitly create axes with the "**axes**" or "**subplot**" functions.

MATLAB Graph

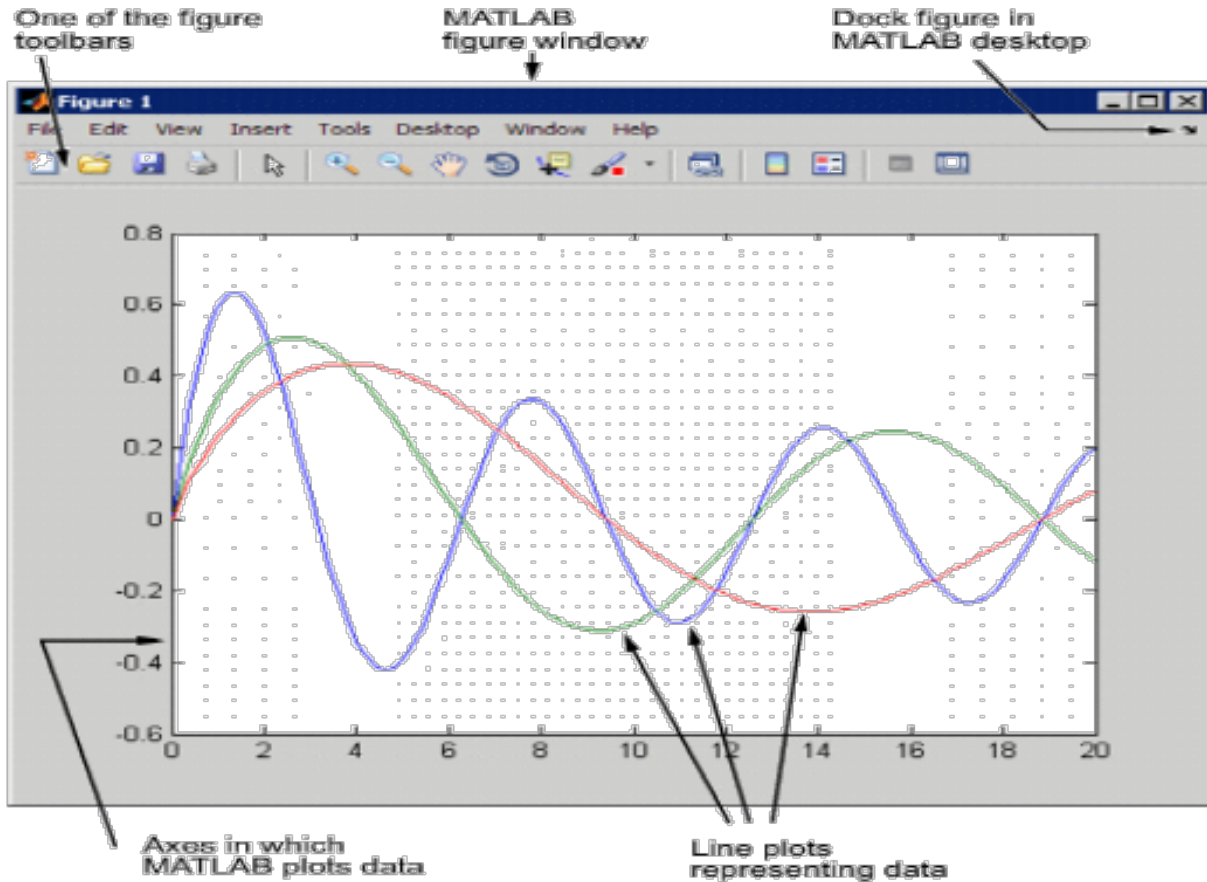
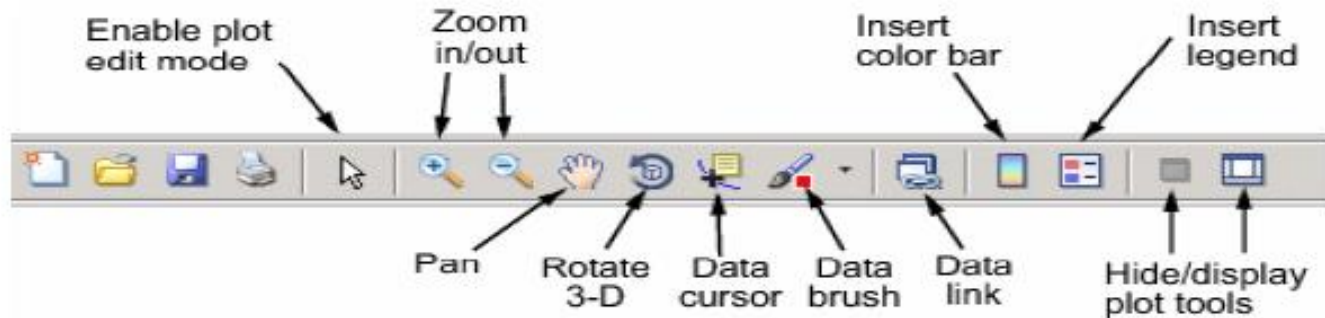
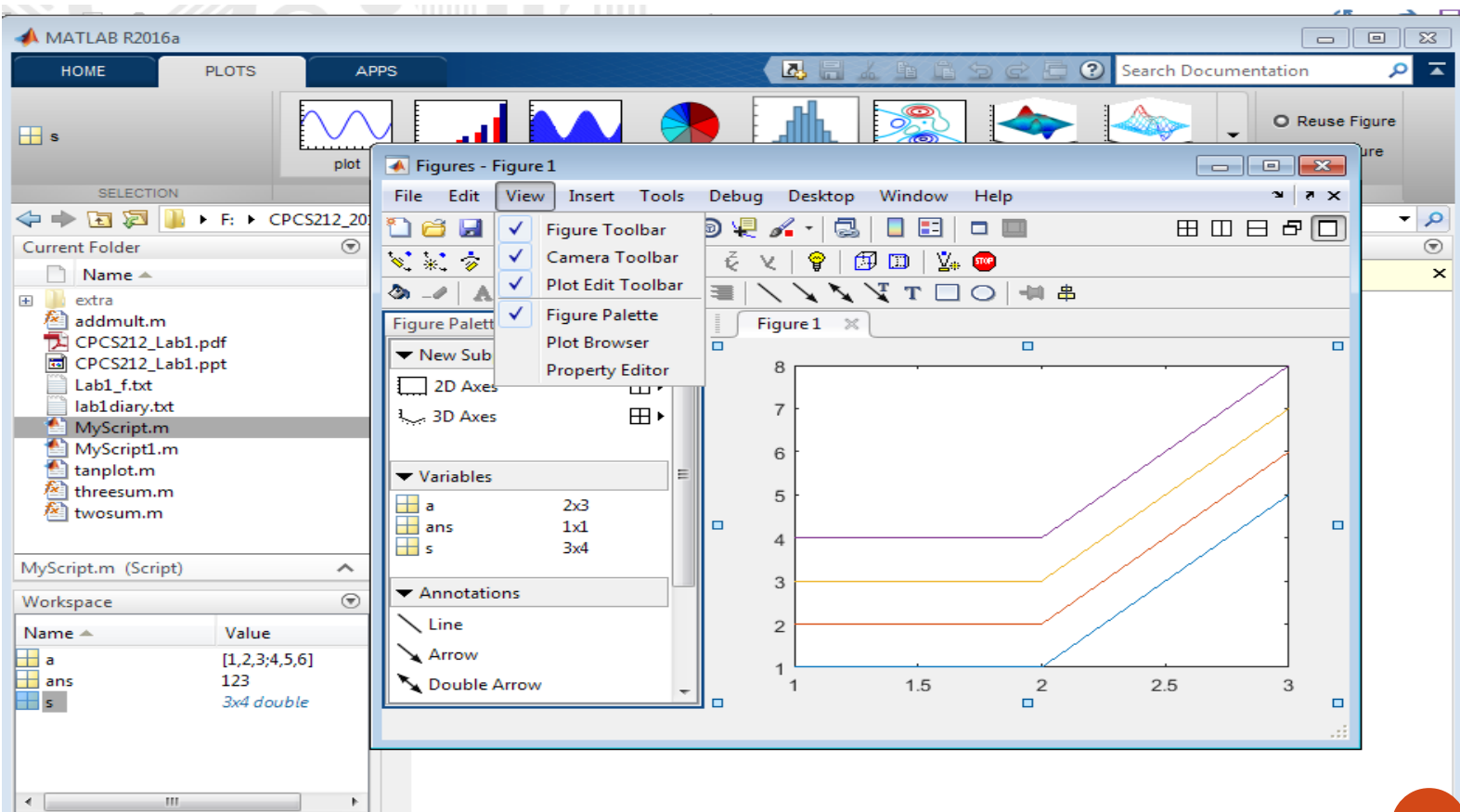


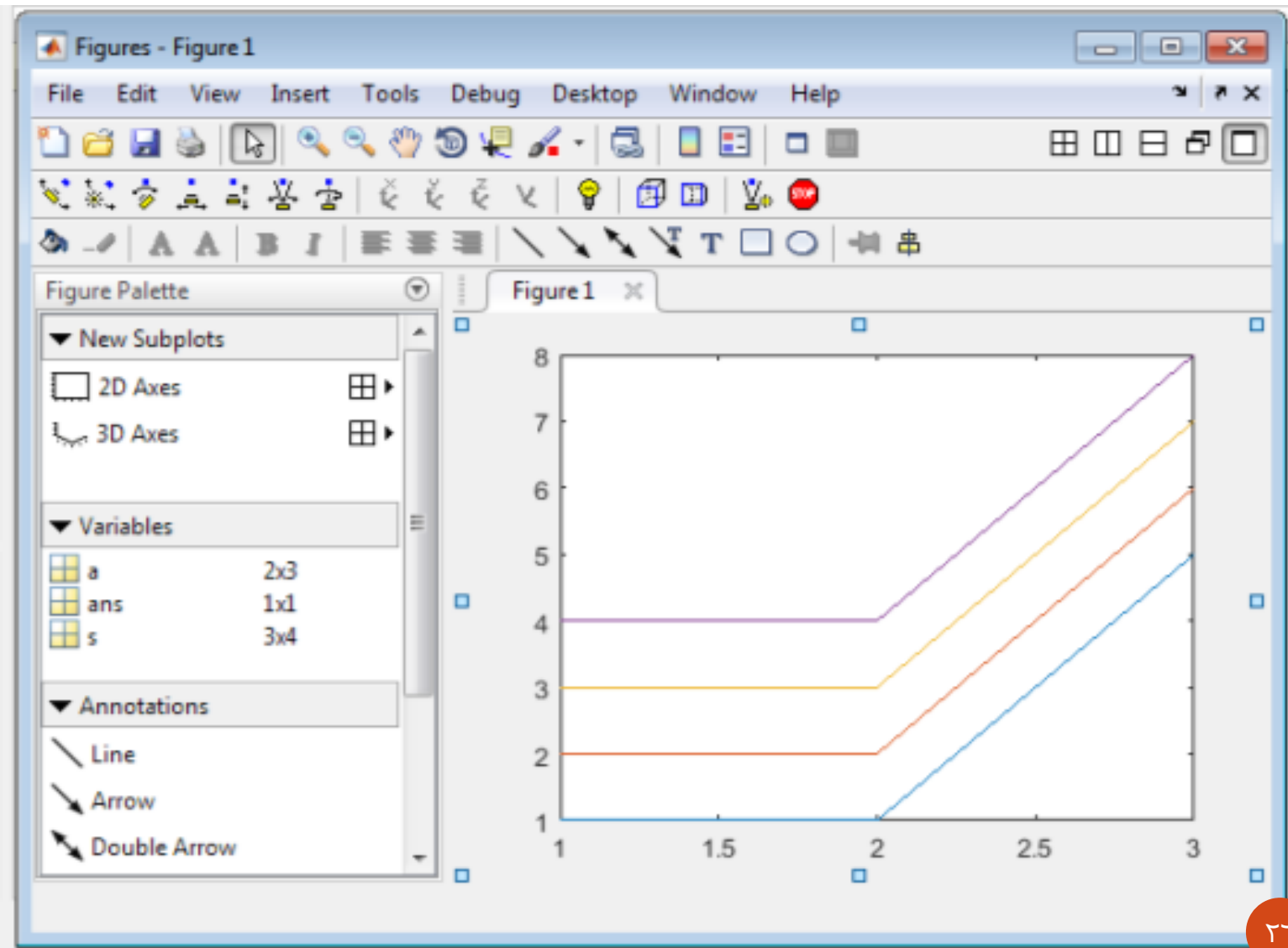
Figure Toolbars

- Figure toolbars provide shortcuts to access commonly used features. These include operations such as saving and printing, plus tools for interactive zooming, panning, rotating, querying, and editing plots. The following picture shows the features available from this toolbar



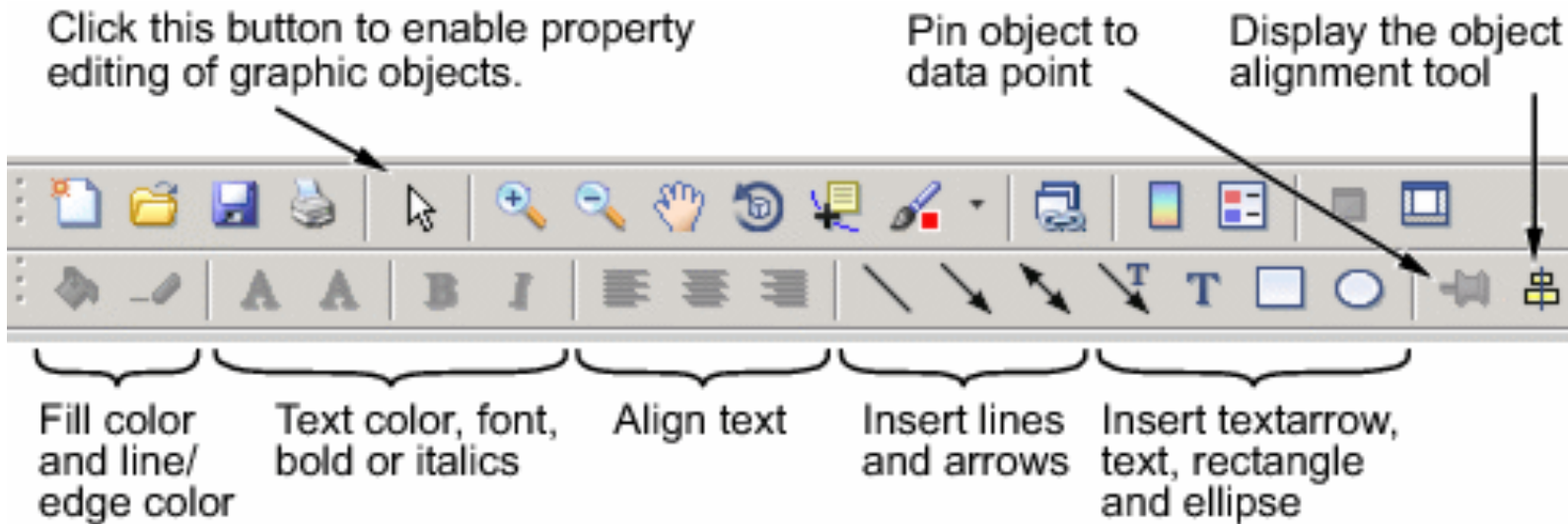
You can enable other toolbars from the View menu:





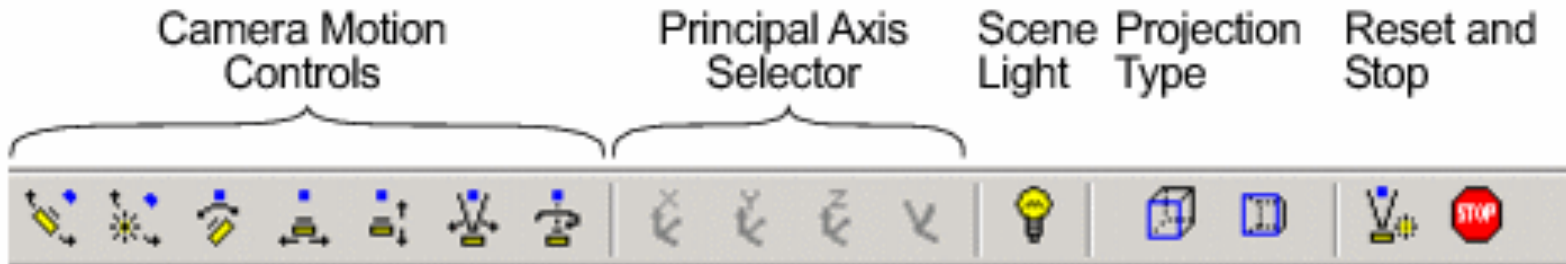
Plot Edit Toolbar

- Use for annotation and setting object properties.

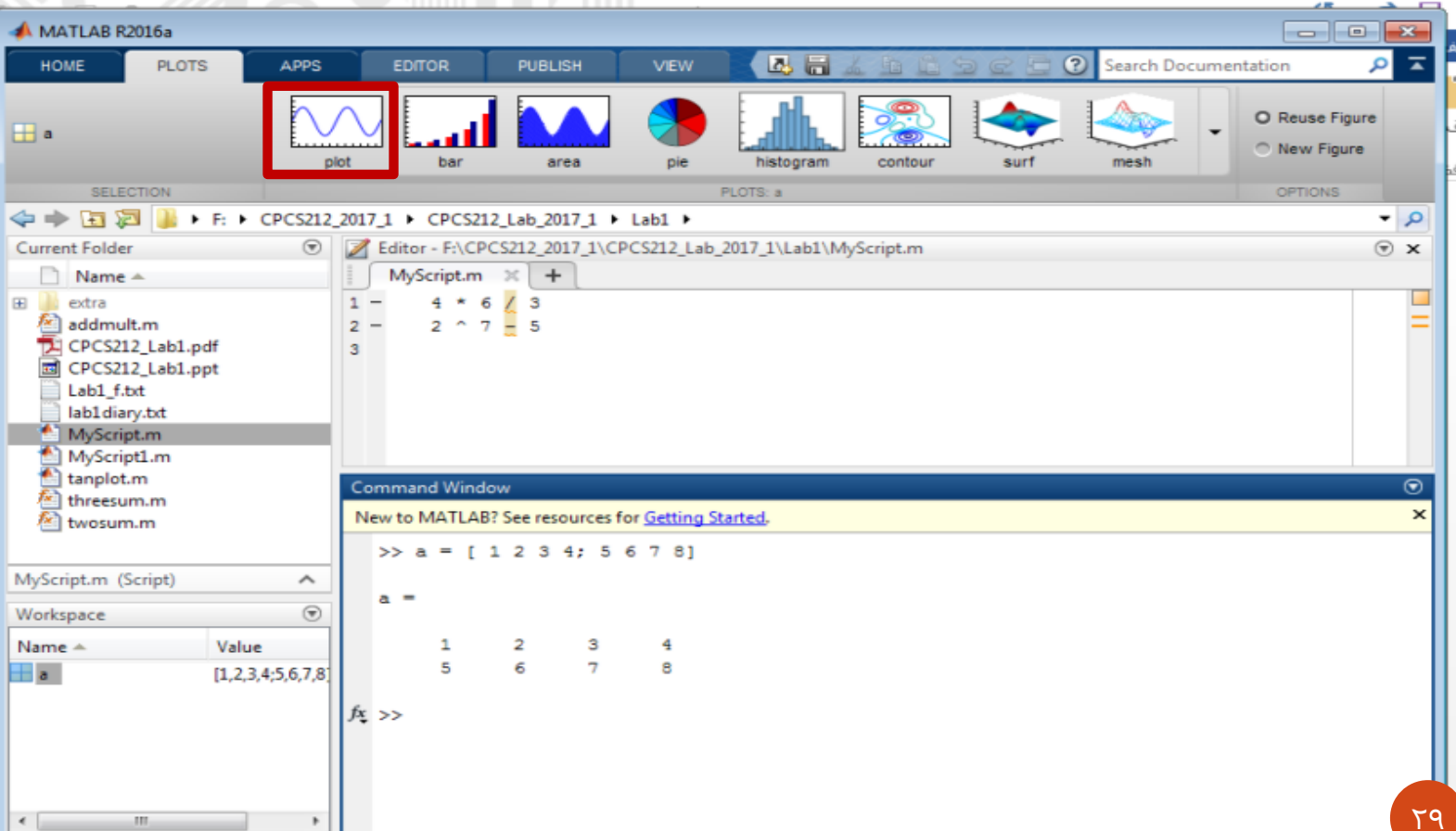


Camera Toolbar

- Use for manipulating 3-D



Plotting



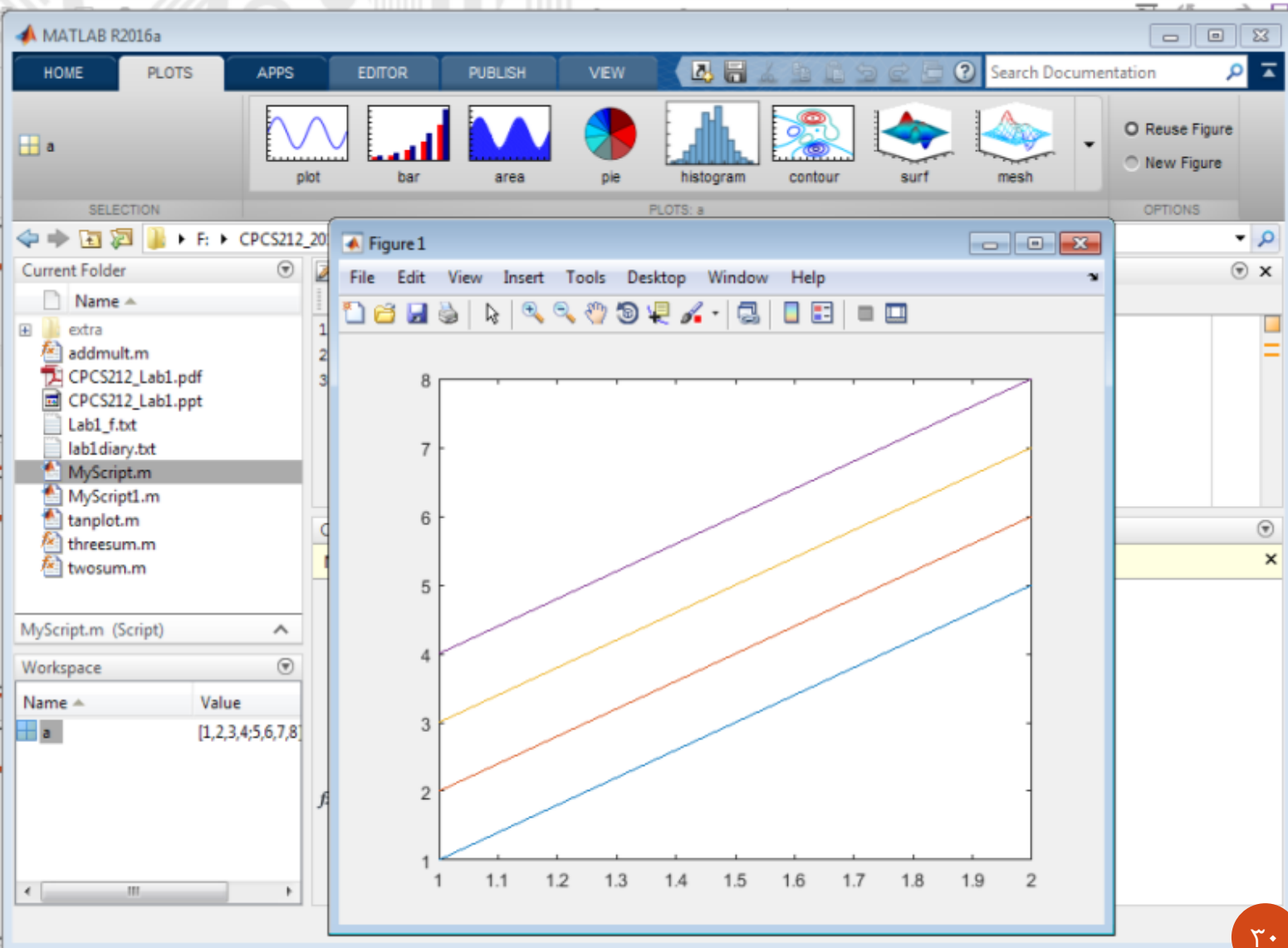
The image displays the MATLAB R2016a software interface, illustrating the process of plotting data. The top menu bar includes HOME, PLOTS, APPS, EDITOR, PUBLISH, and VIEW. The PLOTS tab is active, showing a toolbar with various plotting functions: plot (highlighted with a red box), bar, area, pie, histogram, contour, surf, and mesh. The plot function icon shows a sine wave. Below the toolbar, the current folder is set to F:\CPCS212_2017_1\CPCS212_Lab_2017_1\Lab1. The editor window shows a script named MyScript.m with the following code:

```
1 - 4 * 6 / 3
2 - 2 ^ 7 - 5
3
```

The Command Window displays the command `>> a = [ 1 2 3 4; 5 6 7 8]` and the resulting matrix `a`:

	1	2	3	4
1	1	2	3	4
5	5	6	7	8

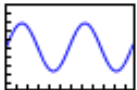
The Workspace window shows the variable `a` with the value `[1,2,3,4;5,6,7,8]`.



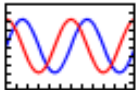
Plotting

Line Graphs

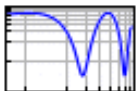
`plot`



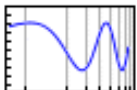
`plotyy`



`loglog`

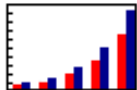


`semilogx`

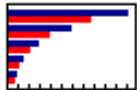


Bar Graphs

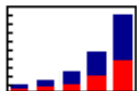
`bar` (grouped)



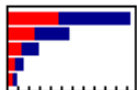
`barh` (grouped)



`bar` (stacked)

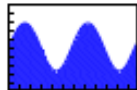


`barh` (stacked)



Area Graphs

`area`



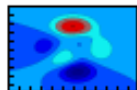
`pie`



`fill`

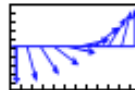


`contourf`

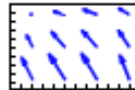


Direction Graphs

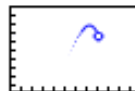
`feather`



`quiver`



`comet`



Radial Graphs

`polar`



`rose`



`compass`

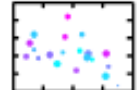


`ezpolar`

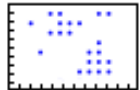


Scatter Graphs

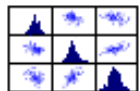
`scatter`



`spy`

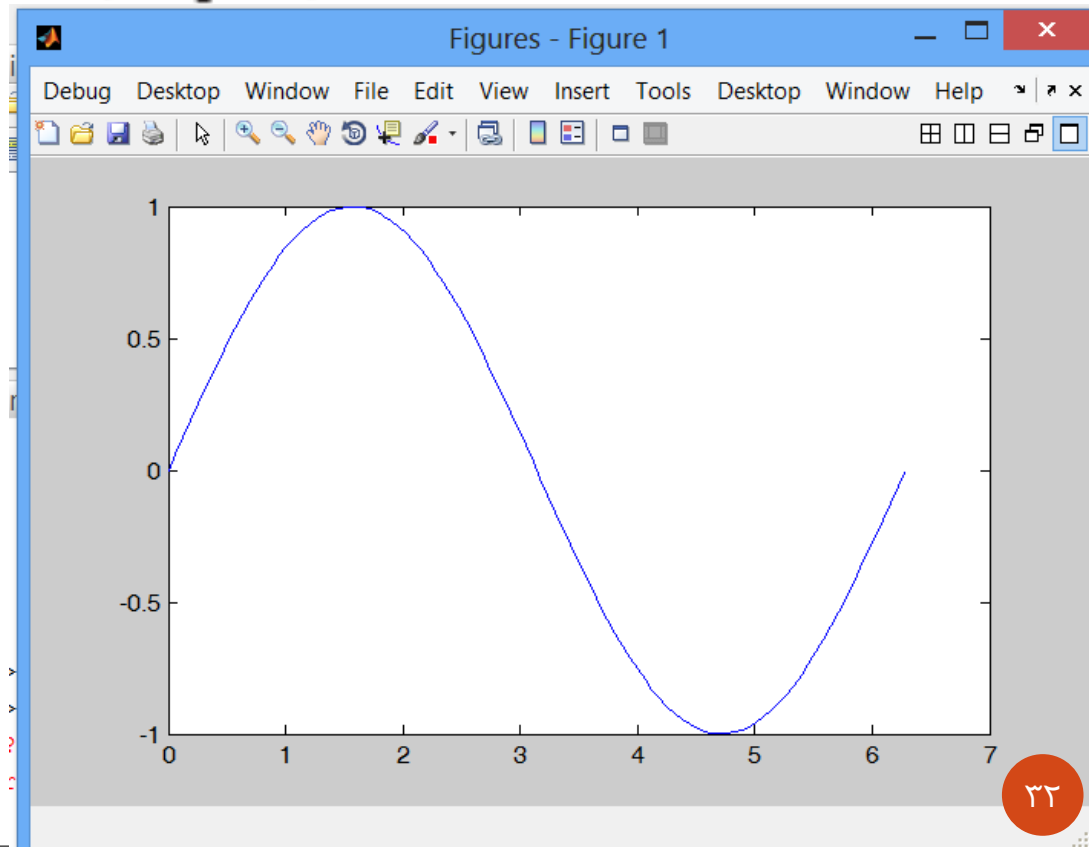


`plotmatrix`



Example 1:

```
>> x = linspace(0,2*pi);  
>> y = sin(x);  
>> plot(x,y);
```



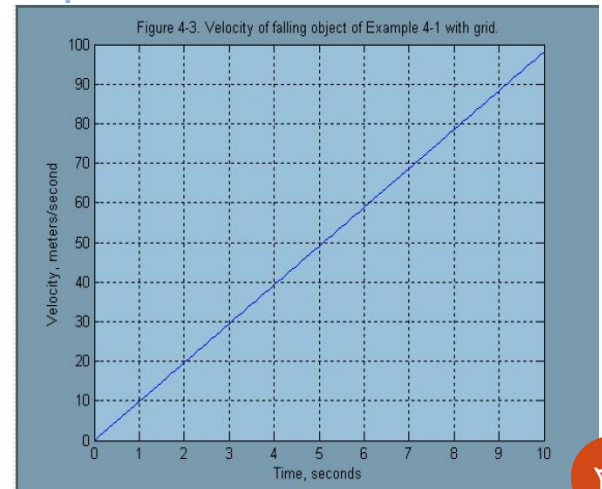
Plot annotation:

Function	Description
<code>title 'name'</code>	Add a title above the plot
<code>xlabel 'name'</code>	Add x-label
<code>ylabel 'name'</code>	Add y-label
<code>legend</code>	Create a legend to identify the symbols
<code>grid</code> or <code>grid on</code> <code>grid minor</code>	Draw grid lines
<code>axis([x_{min} x_{max} y_{min} y_{max}])</code>	Override the axis limits

Example 2

- When air resistance is ignored, the velocity (s/m) of an object falling from rest is: $v = 9.8t$
- Use MATLAB to plot the velocity over a time interval from 0 to 10 s.

```
>> t = 0 : 0.1 : 10;  
>> v = 9.8 * t;  
>> plot(t , v)  
>> xlabel('Times , seconds')  
>> ylabel('velocity , meters/seconds')  
>> title('velocity of falling objec')  
>> grid
```



Line and symbol types:

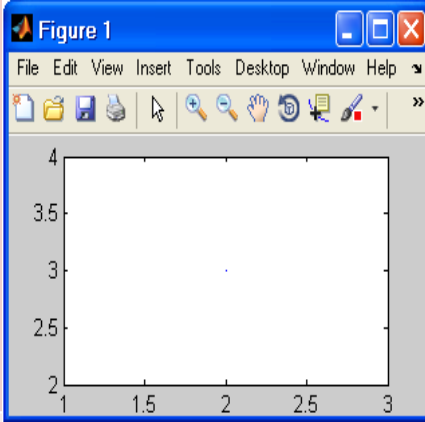
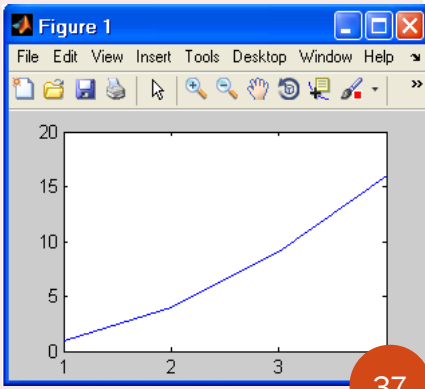
- The curves for a data set are drawn from combinations of color, symbol(marker) and line types in the following table:

Color		Symbols				Line	
y	yellow	.	point	^	triangle (up)	-	solid
m	magenta	o	circle	<	triangle (left)	:	dotted
c	cyan	x	x-mark	>	triangle (right)	-.	dashdot
r	red	+	plus	p	pentagram	--	dashed
g	green	*	star	h	hexagram		
b	blue	s	square				
w	white	d	diamond				
k	black	v	triangle (down)				

Example 3

- Put yellow circles at the data points:
- `>> plot(t , v, 'yo')`
- plot a red dashed line with no symbol:
- `>> plot(t , v , 'r--')`
- Put black diamonds at each data point and connect
- the diamonds with black dashed lines
- `>> plot(t , v , 'kd--')`

Plotting

Matlab expression	Description	Example	output
<code>Plot(x,y)</code> <code>x,y</code> are scalar values	Plot one point on the figure	<code>Plot(2,3)</code>	
<code>Plot(x,y)</code> <code>x,y</code> are vectors	Plot line passing through matrix <code>x,y</code>	<code>Plot([1 2 3 4] , [1 4 9 16])</code>	

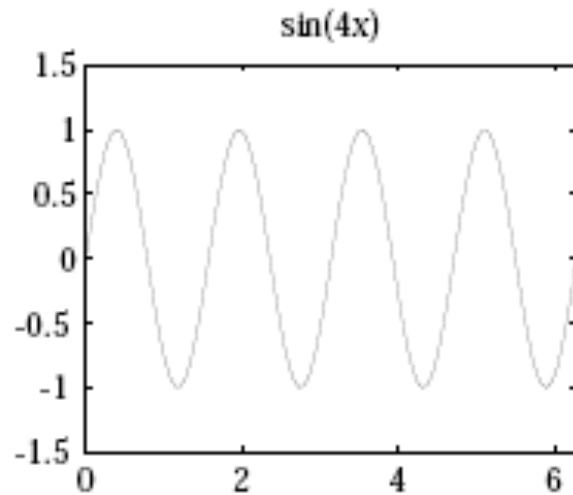
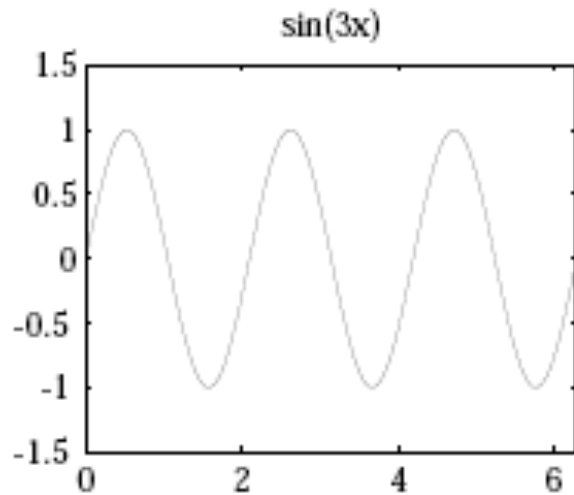
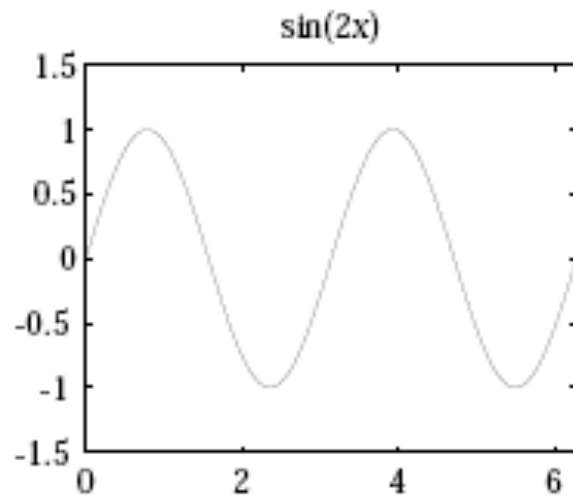
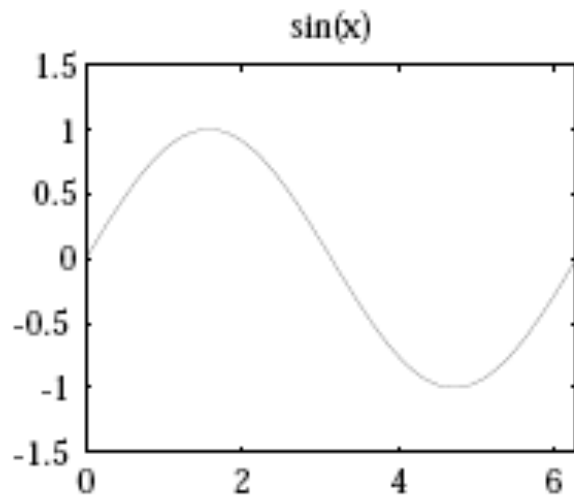
Select a plot region

- The **subplot** function is used to create a matrix of plots in a single figure window.

```
subplot(nrows,ncols,thisPlot)
```

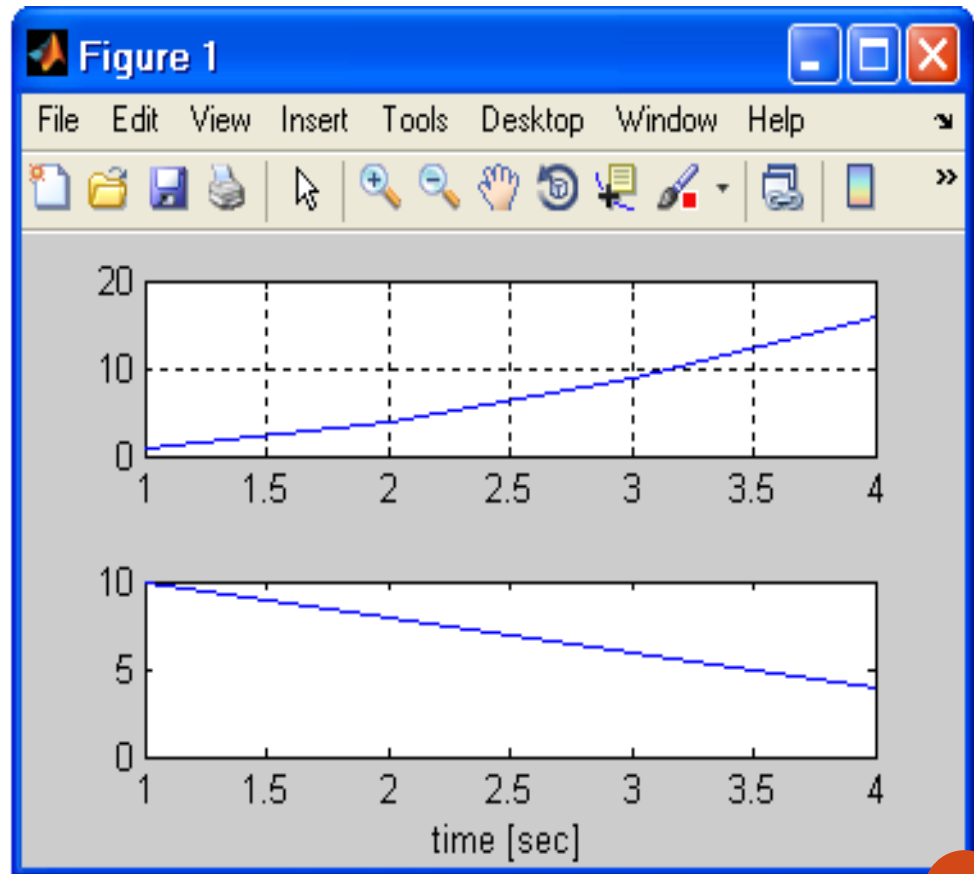
Example 4:

```
>> x = linspace(0,2*pi);  
>> subplot(2,2,1);  
>> plot(x,sin(x));    axis([0 2*pi -1.5 1.5]);    title('sin(x)');  
  
>> subplot(2,2,2);  
>> plot(x,sin(2*x));  axis([0 2*pi -1.5 1.5]);  title('sin(2x)');  
  
>> subplot(2,2,3);  
>> plot(x,sin(3*x));  axis([0 2*pi -1.5 1.5]);  title('sin(3x)');  
  
>> subplot(2,2,4);  
>> plot(x,sin(4*x));  axis([0 2*pi -1.5 1.5]);  title('sin(4x)');
```



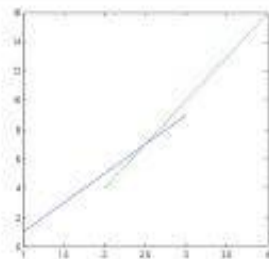
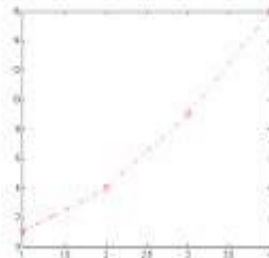
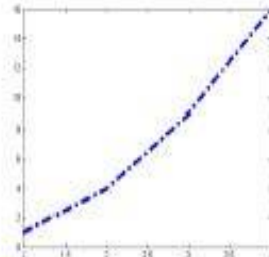
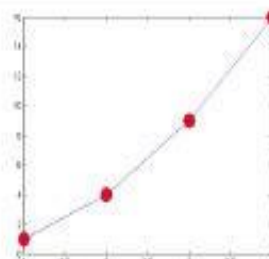
Example 5

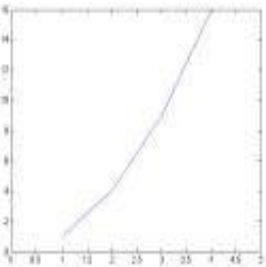
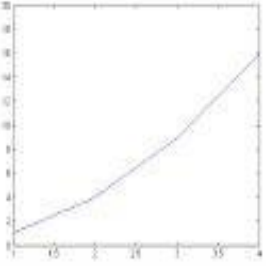
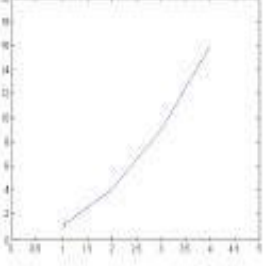
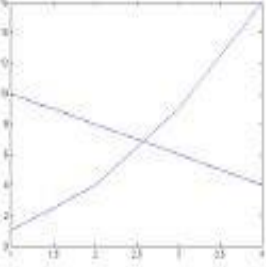
```
>> subplot(211)
plot([1 2 3 4],[1 4 9 16])
subplot(212)
plot([1 2 3 4],[10 8 6 4])
subplot(211)
grid
subplot(212)
xlabel('time [sec]')
>>
```

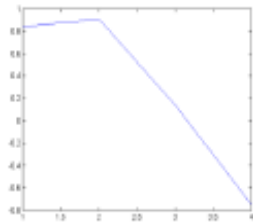
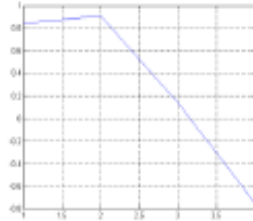
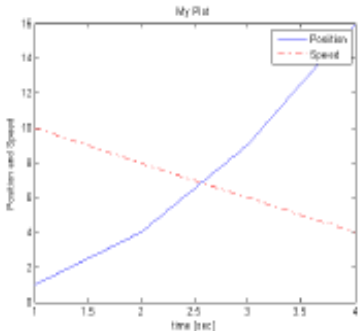
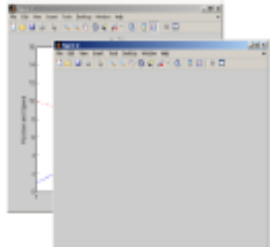


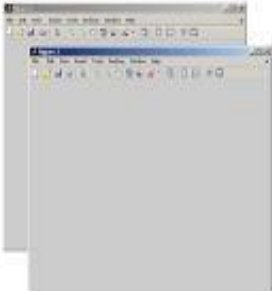
Lab 3

Plotting Activities

<code>plot(x,y)</code> (x,y are matrices)	Plot lines passing through Markers for every columns (x y)	<pre>>> plot([1 2;3 4],[1 4;9 16])</pre>	
<code>plot(x,y,s)</code> (s is a string of 3 characters)	Plot with format s s=cml c: color (b blue, g green, r red, y yellow) m: marker (., o, x, +, *, v, ^, <, >, s square) l: line (-, :, --, --, (none) no line)	<pre>>> plot([1 2 3 4],[1 4 9 16],'ro-')</pre>	
<code>plot(x,y,s1,v1,s2,v2,...)</code> (s1 string, vi value)	Plot with properties si having value vi	<pre>>> plot([1 2 3 4],[1 4 9 16],'LineStyle','-','LineWidth',5)</pre>	
		<pre>>> plot([1 2 3 4],[1 4 9 16],'Marker','o','MarkerSize',15,'MarkerFaceColor',[1 0 0])</pre>	

<code>xlim([xmin xmax])</code>	X-Axis Scaling	<pre>>> plot([1 2 3 4],[1 4 9 16]) >> xlim([0 5])</pre>	
<code>ylim([xmin xmax])</code>	Y-Axis Scaling	<pre>>> plot([1 2 3 4],[1 4 9 16]) >> ylim([0 20])</pre>	
<code>axis([xmin xmax ymin ymax])</code>	XY-Axis Scaling	<pre>>> plot([1 2 3 4],[1 4 9 16]) >> axis([0 5 0 20])</pre>	
<code>hold on</code> or <code>hold</code> <code>hold off</code>	Adds New Plots to Current Plot Replaces Current Plot by New Plot	<pre>>> plot([1 2 3 4],[1 4 9 16]) >> hold on >> plot([1 2 3 4],[10 8 6 4])</pre>	

		<pre>>> hold off >> plot([1 2 3 4],sin([1 2 3 4]))</pre>	
grid on grid off	Adds Grid Lines to Current Axes Removes Grid Lines from Current Axes	<pre>>> plot([1 2 3 4],sin([1 2 3 4])) >> grid</pre>	
xlabel('text')	Adds text beside the X-axis	<pre>>> plot([1 2 3 4],[1 4 9 16]) >> hold on >> plot([1 2 3 4],[10 8 6 4],'r-.') >> xlabel('time [sec]') >> ylabel('Position and Speed') >> title('My Plot') >> legend('Position','Speed')</pre>	
ylabel('text')	Adds text beside the Y-axis		
title('text')	Adds text at the Top		
legend(s1,s2,s3,...) (si are strings)	Puts a legend using s1,s2,s3,... as Labels for Every Plot		
figure	Creates a New Figure Window and Gives it a New Number	>> figure	

<code>figure(n)</code> (<i>n</i> is an integer)	Creates a New Figure Number <i>n</i> Activates Figure Number <i>n</i>	<pre>>> figure(2) >> figure(5) >> figure(7) >> figure(5)</pre>	
<code>close(n)</code> (<i>n</i> is an integer)	Closes Figure Number <i>n</i>	<pre>>> close(7)</pre>	
<code>close all</code>	Closes all Figures	<pre>>> close all</pre>	
<code>subplot(r,c,n)</code> (<i>r,c,n</i> are integers)	Creates rxc New Sub-Figures on the Activated Figure and Activates Sub-Figure Number <i>n</i> Activates Sub-Figure Number <i>n</i>	<pre>>> subplot(211) >> plot([1 2 3 4],[1 4 9 16]) >> subplot(212) >> plot([1 2 3 4],[10 8 6 4]) >> subplot(211) >> grid >> subplot(212) >> xlabel('time [sec]')</pre>	